

PS2 — Hyperlocal Flood Prediction & Response

Full Architecture & Build Plan (for Claude Code execution)

1. Reframing the Problem

The problem statement asks for a **prototype**, not a production warning system (explicitly: *"No real-time deployment or public warning system is required"*). That constraint is a gift — it means you can prioritize:

- A **convincing, data-backed risk model** over a bulletproof pipeline
- A **beautiful, demo-ready map UI** over infrastructure scale
- **Explainability + uncertainty communication** — this is explicitly graded ("communicate confidence," "explain factors")
- **Adaptivity** — the model/dashboard should visibly update as new data comes in, not just show one static snapshot

Your idea to go **Country** → **State** → **Hyperlocal** is good and matches how CWC/NDMA actually structure flood response (national → state → district → station), so lean into that — it also make the judges instantly recognize the UX pattern from real government tools like FloodWatch India.

2. Data Sources (real, checked, current as of Sept 2026)

No single Indian source gives you a clean, free, instant REST API — this is the actual bottleneck every team at this hackathon will hit. Here's the honest map of what's usable, ranked by integration effort:

Tier 1 — Free, instant, no key, use these first

Source	What it gives	Access
Open-Meteo Weather API	Hyperlocal rainfall (hourly/daily), forecasts up to 16 days, historical archive back decades	https://api.open-meteo.com/v1/forecast — no key, no signup
Open-Meteo Flood API (GloFAS)	Simulated river discharge (m ³ /s) for any coordinate, 1984–present + ~7 month forecast, ensemble members for uncertainty	https://flood-api.open-meteo.com/v1/flood — no key
Open-Meteo Geocoding API	Resolve Indian city/town names → coordinates	no key
OpenStreetMap / Overpass API	Drainage lines, water bodies, rivers, low-lying land use, elevation-adjacent tags — good for a "drainage density" feature	free, rate-limited
India state/district GeoJSON boundaries	For your choropleth India map	public GitHub repos (e.g. geohackindia)

Open-Topography / Copernicus DEM (30m)	Elevation → derive slope, low-lying flood-prone zones	free, API key via signup (instant)
--	---	------------------------------------

This tier alone is enough to build a **fully working, live, real-data prototype** with zero waiting on approvals. Start here.

Tier 2 — Free but needs a waitlist/registration (start these on day 1, use if approved)

Source	What it gives	Access
Google Flood Hub / Flood Forecasting API	ML-based riverine flood forecasts, 7-day lead time, covers India, CC BY 4.0, genuinely free	requires waitlist approval + Google Cloud API key — apply immediately, may not arrive in time
NASA GPM IMERG (GES DISC)	Satellite-derived rainfall, near-real-time, high resolution	free Earthdata login (instant approval), slightly more integration work

Tier 3 — Real Indian government sources, but portal/dashboard-only (no open REST API)

Use these for **credibility, ground-truth validation, and historical training data**, not as your live pipeline:

- **CWC Advisory Flood Forecast** (`aff.india-water.gov.in`) and the new **C-FLOOD** system — 332 forecast stations, 20 river basins, publishes 5–7 day advisories. No public API, but downloadable data/screens you can scrape sparingly or cite.
- **India-WRIS / Water Data Online** (`indiawriss.gov.in`, `wdo.indiawriss.gov.in`) — rainfall, river level, reservoir, groundwater time series. Portal + downloadable Excel/CSV, not a live API.
- **IMD** — no public rainfall/nowcast API; some datasets appear on `data.gov.in`. Use IMD's published **color-coded alert protocol (Green/Yellow/Orange/Red)** as your classification standard even if you don't pull live IMD data — this is the single most important "make it feel official" move.
- **NDMA** — mostly SOPs, historical disaster reports, guidelines (PDF). Good for a "protocol alignment" section in your app (e.g. showing which NDMA response tier a risk level maps to) and for historical incident data to validate your model against.
- **Bhuvan (ISRO/NRSC)** — flood inundation layers, satellite imagery, some WMS layers usable in a map as an overlay.

Tier 4 — Optional / Hugging Face

There's no ready-made "India flood" model on Hugging Face worth relying on. Better uses of HF here:

- A small pretrained time-series/tabular model (e.g. lightweight gradient boosting or LSTM you fine-tune on GloFAS + rainfall features yourself, hosted for free inference)
- A small instruction-tuned text model to auto-generate the **plain-language explanation** of why a location is flagged ("river discharge 40% above seasonal median + 92mm rain in 24h") — this maps directly to the "explain the factors" requirement
- Skip HF for anything claiming to be a ready flood predictor — none exist that are trustworthy; build your own scoring model instead (see §4)

Practical recommendation: build the whole pipeline on **Open-Meteo (weather + GloFAS flood)** as your live backbone since it needs zero approval and already covers all of India at 5km resolution. Layer in Google Flood Hub if the waitlist clears in time. Use CWC/India-WRIS/NDMA numbers as validation, citations, and for calibrating your alert thresholds so judges see it's grounded in real Indian protocol, not invented numbers.

3. System Architecture

flowchart TD

```

subgraph Ingestion["Data Ingestion Layer (scheduled + on-demand)"]
    A1[Open-Meteo Weather API]
    A2[Open-Meteo Flood/GloFAS API]
    A3[OSM Overpass - drainage/water bodies]
    A4[DEM - elevation/slope]
    A5[Static: CWC/NDMA historical flood events]
end

subgraph Store["Data Layer"]
    B1[(PostGIS - locations, time series)]
    B2[(Feature Store - engineered features per cell/city)]
end

subgraph Engine["Risk Engine"]
    C1[Feature Engineering]
    C2[Risk Scoring Model - ML + rules hybrid]
    C3[Temporal Projection - risk evolution over next 24-72h]
    C4[Uncertainty Estimation - ensemble spread / quantiles]
    C5[Explainability - top contributing factors]
    C6[Alert Classifier - Green/Yellow/Orange/Red]
end

subgraph API["Backend API (FastAPI)"]
    D1[/GET /country/summary/]
    D2[/GET /state/:id/risk/]
    D3[/GET /location/:lat/:lon/detail/]
    D4[/GET /location/:id/timeline/]
    D5[/POST /refresh/]
end

subgraph Frontend["Frontend (React + Map)"]
    E1[India Map - state choropleth]
    E2[State Drilldown - district/city markers]
    E3[Hyperlocal Detail Panel - risk, confidence, factors, timeline]
    E4[Risk Evolution Slider]
    E5[Live "last updated" + refresh indicator]
end

A1 --> B1
A2 --> B1
A3 --> B1
A4 --> B1
A5 --> B1
B1 --> C1 --> B2
B2 --> C2 --> C3 --> C4 --> C5 --> C6
--

```

```

C6 --> D1
C6 --> D2
C5 --> D3
C3 --> D4
Engine --> D5
D1 --> E1
D2 --> E2
D3 --> E3
D4 --> E4
D5 --> E5

```

Why this shape satisfies the stated requirements

- **"Handle changing conditions, not a static plan"** → the ingestion layer is scheduled (e.g. every 1–3 hours) and the risk engine recomputes from scratch each run; the frontend shows a live "last updated" timestamp and a manual refresh, proving the system adapts rather than hardcoding one scenario.
- **"Estimate how risk may evolve over time"** → C3 produces a 24h/48h/72h projected risk trajectory per location using the GloFAS ensemble forecast + rainfall forecast, shown as the Risk Evolution Slider.
- **"Communicate confidence"** → C4 turns GloFAS ensemble spread (min/median/max discharge) and forecast lead-time into a simple Low/Medium/High confidence badge, not just a number.
- **"Explain factors"** → C5 outputs a ranked list like: *rainfall (24h): 68/100 · river discharge trend: 82/100 · elevation/drainage: 40/100* — this is what makes the demo feel intelligent instead of a black box.

4. Risk Model Design (keep it explainable, not a black box)

For a hackathon, a **hybrid rule-weighted + lightweight ML model** beats a deep learning black box — it's faster to build, easier to explain live to judges, and you can defend every number.

Core features per location (city/district centroid or grid cell):

1. Rainfall last 24h / 72h / 7-day cumulative (Open-Meteo)
2. Forecast rainfall next 24h/48h/72h
3. River discharge current vs. seasonal median (GloFAS)
4. River discharge trend (rising/falling, rate of change)
5. GloFAS ensemble spread (→ uncertainty)
6. Elevation + local slope (DEM)
7. Distance to nearest river/water body (OSM)
8. Drainage density in area (OSM waterway/drain tag density)
9. Historical flood frequency at/near this location (from NDMA/CWC historical events, static dataset you compile once)

Scoring approach:

- Normalize each feature 0–100
- Weighted sum → base risk score (weights tuned against historical flood events as a sanity check — e.g. does your score spike for known 2023–2025 flood dates in Assam/Bihar/Chennai/Mumbai?)

- Wrap in a simple **XGBoost or logistic regression classifier** trained on historical event labels (flooded/not flooded) if you have time — even a small, honestly-labeled dataset of ~50-100 historical events beats an unvalidated rule set alone
 - Map final score → **4-tier alert (Green/Yellow/Orange/Red)** aligned to IMD's real color convention so the mapping feels official
 - Confidence badge derived from (a) GloFAS ensemble spread and (b) forecast lead time (closer-in-time = higher confidence)
-

5. Frontend / UX Plan

Screen 1 — Country View

- India choropleth map, colored by state-level aggregated risk (max or 90th-percentile risk among that state's monitored cities)
- Top strip: count of Red/Orange/Yellow/Green locations nationally, last-updated timestamp
- Click a state → Screen 2

Screen 2 — State View

- Zoomed map of the state with city/district markers, colored by risk tier, sized by population or impact potential
- Sidebar list sortable by risk score
- Click a marker → Screen 3

Screen 3 — Hyperlocal Detail

- Risk score + tier badge + confidence badge
- Factor breakdown (bar chart: rainfall, river discharge, drainage, historical frequency)
- 72-hour risk evolution chart (line with uncertainty band)
- Plain-language explanation (auto-generated, e.g. via a small LLM call or templated text)
- Raw data snapshot (rainfall mm, discharge m³/s) with source attribution

Design direction: dark map base with a clear semantic color scale (green→yellow→orange→red), card-based detail panel, subtle animated transitions on data refresh. Use Mapbox GL JS or Leaflet + a vector India GeoJSON; D3 or Recharts for the timeline/factor charts.

6. Suggested Tech Stack

- **Backend:** Python + FastAPI (async, easy to call multiple external APIs concurrently) or Node/Express if your team prefers JS end-to-end
- **Database:** PostgreSQL + PostGIS extension (spatial queries: nearest station, state boundaries) — SQLite fine for a pure hackathon MVP if time is tight
- **Cache/scheduler:** Redis + APScheduler (or a simple cron) to refresh data every 1–3 hours without hammering APIs
- **ML:** scikit-learn / XGBoost for the risk classifier; pandas/numpy for feature engineering
- **Frontend:** React + Vite, Mapbox GL JS or Leaflet, Recharts or D3 for charts, Tailwind for styling

- **Deployment (demo-only, no real-time public warning needed):** can run entirely locally or on a single free-tier host (Render/Railway/Vercel) since there's no requirement for production-grade uptime

7. Build Order (time-boxed for a hackathon)

Phase	Time	Deliverable
0. Setup	1–2h	Repo scaffold, API key registrations (Open-Meteo needs none; start Google Flood Hub waitlist immediately in parallel), pick ~15-20 representative Indian cities/flood-prone districts to seed as your monitored location set
1. Data ingestion MVP	3–4h	Working pull of rainfall + GloFAS discharge for all seeded locations into your DB, on a schedule
2. Risk engine	3–4h	Feature engineering + weighted scoring + tier classification + confidence + factor breakdown, validated against 3–5 known historical flood events
3. Backend API	2h	Endpoints for country/state/location summaries + timeline
4. Frontend Screen 1+2	3–4h	India map + state drilldown, wired to live API
5. Frontend Screen 3	2–3h	Hyperlocal detail panel, risk evolution chart, explanation text
6. Polish + adaptivity proof	2h	Visible refresh/last-updated indicator, trigger a manual refresh live during demo to prove it's not static, README + architecture diagram
Stretch	remaining time	Google Flood Hub integration if approved, Bhuvan satellite overlay, historical event replay mode ("show me what this would've predicted before the 2023 X flood")

The **historical event replay** stretch goal is worth prioritizing if you have spare time — running your model against a real past flood date and showing it would have flagged the area is far more convincing to judges than any live demo, since it doesn't depend on rain happening during your presentation.

8. Prompt Block — paste this to Claude Code to start execution

Build a full-stack "Hyperlocal Urban Flood Prediction & Response" prototype for India.

TECH STACK:
- Backend: Python FastAPI, PostgreSQL+PostGIS (fallback SQLite if simpler), APScheduler for per
- Frontend: React + Vite, Mapbox GL JS (or Leaflet) for the India map, Recharts for charts, Tai

- ML: scikit-learn/XGBoost for a risk classifier, pandas/numpy for feature engineering

DATA SOURCES (call these directly, no API keys needed for the first two):

1. Open-Meteo Weather API (<https://api.open-meteo.com/v1/forecast>) - rainfall, forecast
2. Open-Meteo Flood API (<https://flood-api.open-meteo.com/v1/flood>) - GloFAS river discharge +
3. OSM Overpass API - drainage/water body density near a coordinate
4. Static seed file of ~15-20 Indian flood-prone cities/districts with lat/lon (e.g. Mumbai, Chennai)
5. Static historical flood events JSON (a handful of real, dated past flood events per city, for context)

BUILD, IN ORDER:

1. Project scaffold: /backend (FastAPI) and /frontend (React), Docker-compose optional
2. Ingestion module: pulls rainfall + discharge for each seeded location on a schedule (every N minutes)
3. Feature engineering module: computes cumulative rainfall (24h/72h/7d), discharge trend, ensemble model
4. Risk scoring: weighted rule-based score (0-100) blended with a simple trained classifier if available
5. Temporal projection: produce a 24h/48h/72h risk trajectory per location using the forecast data
6. REST API endpoints:
 - GET /api/country/summary (state-aggregated risk for the India map)
 - GET /api/state/{state_id}/locations (all monitored locations + risk in that state)
 - GET /api/location/{id}/detail (score, confidence, factor breakdown, plain-language explanation)
 - GET /api/location/{id}/timeline (72h risk evolution)
 - POST /api/refresh (manually trigger a re-ingestion + re-score, for demo purposes)
7. Frontend:
 - Screen 1: India choropleth map by state risk, clickable
 - Screen 2: State view with city/district markers colored/sized by risk, clickable, sidebar
 - Screen 3: Location detail panel - risk badge, confidence badge, factor bar chart, 72h risk trajectory
 - Dark map theme, green->yellow->orange->red semantic color scale throughout
8. Add a README with architecture diagram, data source citations, and instructions to run locally

Build this incrementally and show me each phase working before moving to the next.